COMP6841 Security Engineering Project

Defence in Depth Across the Stack

SQL Injection, XSS, Buffer Overflow, and Format String Vulnerabilities Under Three Layers of Protection

En-Mei Chiu (z5727209)

COMP6441/COMP6841 Security Engineering, UNSW, 2026 T2

Code and full evidence: github.com/Amychiu1/comp6841-security-project

Abstract

This project looks at four vulnerability classes from opposite ends of a normal application stack — SQL injection and XSS on the web side, stack buffer overflow and a format string bug on the binary side — and asks the same question of all four: what actually has to be true for a fix to work, not just look like it works? I built three versions of every target (no protection, a weak/naive protection, a proper industry-standard fix) and reused the same attack across all three wherever possible, so any difference in outcome had to come from the defence itself. The pattern held every time: defences that look for a specific known-bad pattern can be talked around once you work out what they're actually checking, while defences that remove the vulnerable operation altogether held up even against payloads built to beat the weaker version. I've also tried to be honest about where the 'proper' fixes still fall short, and reflected on what building and attacking my own vulnerable code actually taught me.

1. Introduction

1.1 Context and Scope

Software gets attacked at every layer — from the page someone's clicking around in, down to raw memory inside a compiled binary. I picked one vulnerability from each end of that range that's still genuinely common: SQL injection and XSS on the web side, a stack buffer overflow and a format string bug on the binary side. The question was the same for all four: what does a defence actually have to do to be real, rather than just look like it's working? I built small, deliberately broken targets for each bug in three versions — no protection, a weak protection a less experienced developer might genuinely think was enough, and a proper industry-standard fix — then attacked all three with the same payloads wherever possible, so any change in outcome could only be down to the defence.

This project was originally proposed as a walkthrough of the OWASP Top 10 web vulnerabilities. After getting feedback that the proposal needed to go beyond the normal scope of the course, I ended up redesigning it to combine the OWASP-side web vulnerabilities (SQL injection and XSS) with two binary exploitation classes that sit completely outside OWASP's scope — buffer overflow and format string bugs — and testing all four with the same three-layer approach described below. It ended up being a much bigger technical range than my original plan, but the underlying question stayed the same across both halves.

1.2 Problem Statement and Objectives

A pattern that keeps showing up in real security incidents is that a lot of breaches aren't caused by some exotic zero-day — they're caused by a defence that was only ever built to stop the one attack the developer already had in mind. What I wanted to show here, with actual working code rather than just an argument, is why blacklist-style defences are structurally weaker than defences that remove the vulnerable class of operation completely, and I wanted to show it across two quite

different technical areas (web logic and native binary memory corruption) so the conclusion isn't just something specific to one language or framework. Concretely, I set out to:

• Build a small web app with an unpatched SQL injection bug and an unpatched stored XSS bug, each tested with a baseline, a naive fix, and a proper fix.

• Build two small C programs — a classic ret2win stack overflow and a format string bug — each compiled or fixed across three layers of increasing protection.

• Attack every layer with the same or a directly comparable payload, and note down where and why each defence succeeded or failed.

• Pull out whatever common pattern connects all four vulnerability classes, and reflect honestly on what I learned, technically and otherwise, from building and then attacking my own vulnerable software.

2. Background and Related Work

2.1 Web Application Vulnerabilities: SQL Injection and XSS

SQL injection and XSS are both still near the top of the list of common web application weaknesses. OWASP's Top 10 puts SQL injection under the broader Injection category, describing the root cause as untrusted data getting concatenated straight into a query instead of being kept separate from it (OWASP Foundation, 2021). The standard fix is a parameterised query, where the query text and the values a user typed are sent to the database as two separate things (OWASP Foundation, n.d.).

XSS happens when untrusted input gets rendered into a page as if it were trusted markup instead of just data. OWASP's XSS Prevention Cheat Sheet (OWASP Foundation, n.d.) points to contextual output encoding, not input filtering, as the fix that holds up, since encoding turns the characters a browser cares about into inert text regardless of the payload's shape.

2.2 Binary Exploitation Vulnerabilities: Buffer Overflow and Format String

Stack-based buffer overflow (CWE-121) happens when a program writes past the end of a fixed-size stack buffer, which classically lets an attacker overwrite the saved return address and redirect execution. Modern toolchains stack several defences together: a canary that notices the overwrite before the function returns, NX/DEP marking the stack non-executable, PIE combined with ASLR randomising where code and data load at runtime, and RELRO locking down the sections used to resolve dynamic symbols.

Format string vulnerabilities (CWE-134) happen when a program hands attacker-controlled data straight to printf as the format argument, instead of a normal data argument alongside a fixed format string. Because printf treats its format string as instructions rather than plain text, %x and %p can read memory never meant to be visible, and %n becomes a genuine arbitrary-write primitive.

2.3 Existing Tools and Related Work

For the binary half I used pwntools to script payloads and parse crash output, and checksec to check which of the four protections (canary, NX, PIE, RELRO) were switched on in each build — standard tools in CTF exercises and real vulnerability research, nothing I built myself. For the web half I ran a small SQLite-backed Flask app locally so I could inspect every stage directly, which felt closer to reviewing actual application logic than running a black-box scanner like sqlmap or OWASP ZAP, even though those tools are really just the automated version of the manual bypasses done here.

3. Methodology and Implementation

Every vulnerability went through the same three-layer process: a baseline with no protection, a weak/naive protection, and a proper, industry-standard fix. Wherever the bug allowed it, I replayed the exact same payload unmodified against every layer, so any change in the result could only be explained by the defence itself.

3.1 Web Component — SQL Injection

3.1.1 Baseline — no protection

The login route built its SQL query by directly concatenating the username and password straight into the query string:

```
query = "SELECT * FROM users WHERE username = '{}' AND password =
'{}'".format(username, password)
```

Since the input goes straight into the query text, anything typed into the form becomes part of the actual SQL command rather than just data sitting inside it. After confirming a normal failed login with random credentials, I tried the classic authentication-bypass payload in the username field: ' OR '1'='1' --. The database has no way to tell 'code' apart from 'data' here — it just sees one long, valid SQL statement, and I wrote part of it myself.

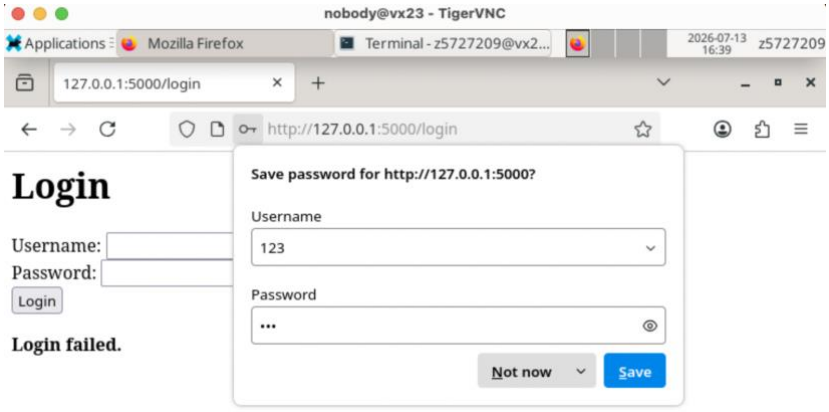

Figure 1: Baseline login with random credentials, failing as expected.

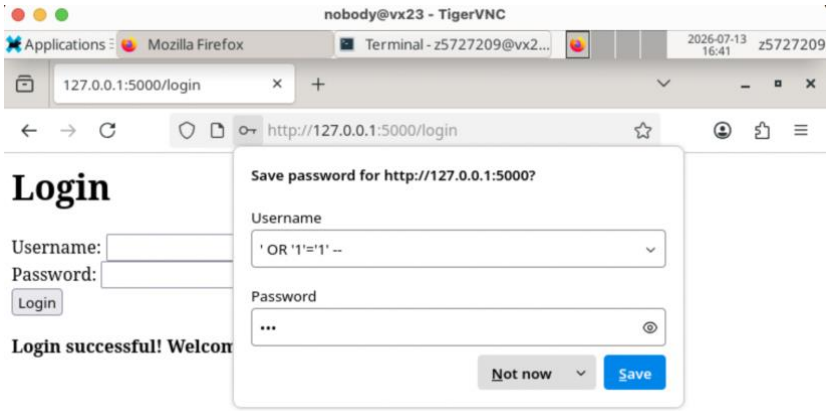

Figure 2: Baseline SQLi bypass — the payload turns the WHERE clause into an always-true condition and logs in as admin without the password.

3.1.2 Weak protection — keyword blacklist

The second version adds a filter that rejects the request if the username or password contains one of a small set of keywords (OR, UNION, --, DROP, SELECT), while underneath, the query is still built the same unsafe way. The original bypass got blocked correctly, but it turned out the filter was case-sensitive and only recognised one style of SQL comment. Changing OR to oR and swapping -- for /* got past it completely: ' oR '1'='1' /*

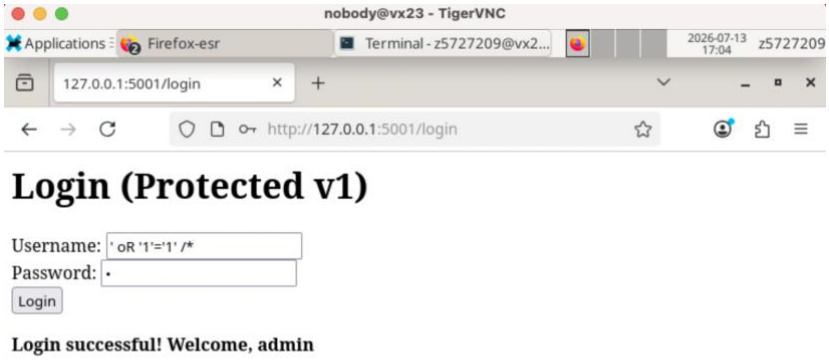

Figure 3: Case variation plus a different comment style bypasses the keyword blacklist entirely — logged in as admin again.

This is a decent example of why blacklists are basically fragile by design — they only defend against the specific text a developer happened to think of, not against what the input actually means, and there's almost always another way to say the same thing.

3.1.3 Proper protection — parameterised query

The real fix swaps the string formatting for a parameterised query, sending the values separately from the query text itself:

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cur.execute(query, (username, password))
```

The database driver now keeps the SQL command and whatever the user typed completely separate at the protocol level, so those values are never parsed as SQL syntax regardless of what characters are in them. Running the exact payload that beat the weak filter now just fails to match any row, while a real login with the correct admin credentials still works fine — the fix gets rid of the vulnerability without breaking anything that was supposed to work.

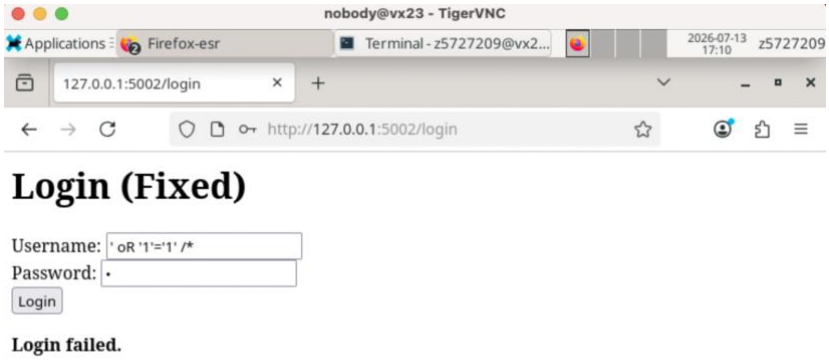

Figure 4: The same bypass payload that worked against the weak filter now simply fails to match any row.

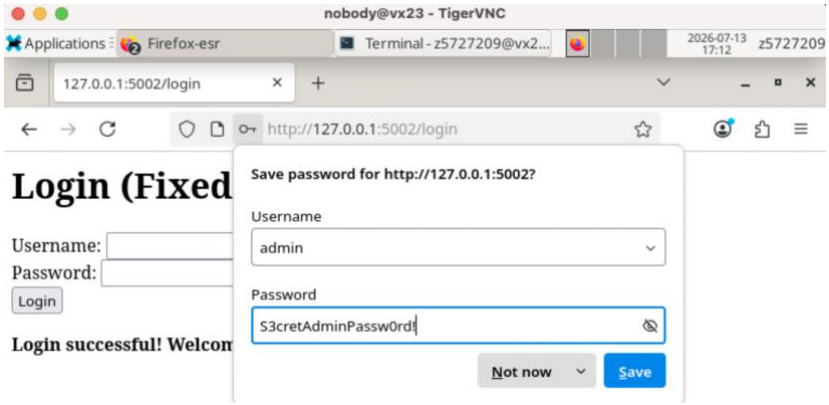

Figure 5: A legitimate login with the correct admin credentials still succeeds.

3.2 Web Component — Cross-Site Scripting (XSS)

3.2.1 Baseline — no protection

The guestbook stores whatever a visitor types and renders it back out using Jinja2's | safe filter, which turns off Flask's default HTML escaping on purpose. After posting a normal comment to check everything worked as expected, I posted <script>alert('XSS')</script>, and it ran as real JavaScript the moment the page loaded. Because | safe is basically telling the template engine 'trust this completely', the browser has no way to tell a visitor's comment apart from code the developer actually wrote.

Figure 6: A normal guestbook comment gets stored and displayed as expected.

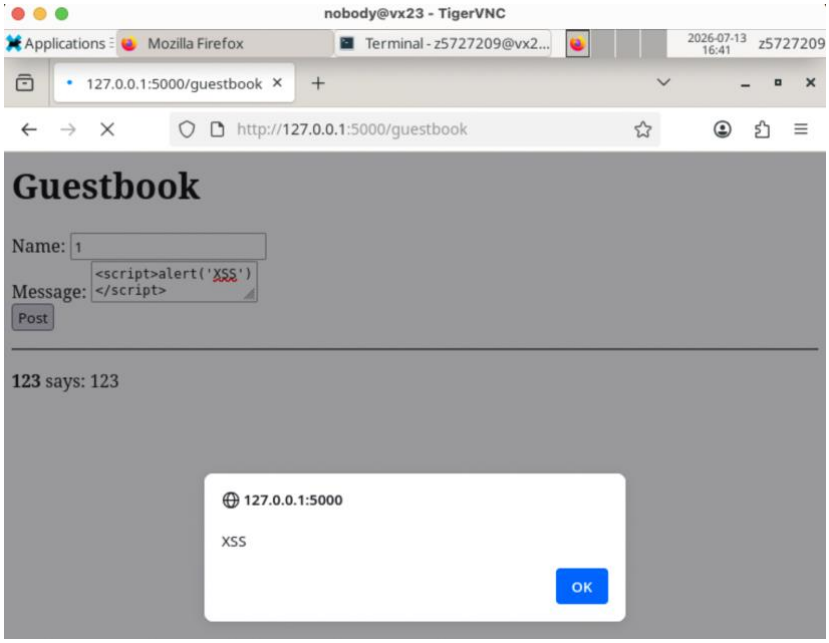

Figure 7: The <script> payload runs as real JavaScript in the browser as soon as the page loads.

Figure 8: A different payload using a broken image and an onerror handler behaves the same way, confirming the bug isn't limited to the <script> tag specifically.

3.2.2 Weak protection — tag stripping

The second version strips out the exact substrings <script> and </script> from any comment before it gets stored:

```
value = value.replace("<script>", "").replace("</script>", "")
```

This blocks the original payload, but only because it's looking for one very specific tag. XSS doesn't need <script> at all — any HTML element that can carry an event handler will do the job. Posting <svg onload=alert(1)> went straight past the filter untouched and ran the moment the browser rendered the SVG element, which mirrors the SQLi bypass almost exactly: the defence is aimed at one known pattern rather than the actual capability being abused.

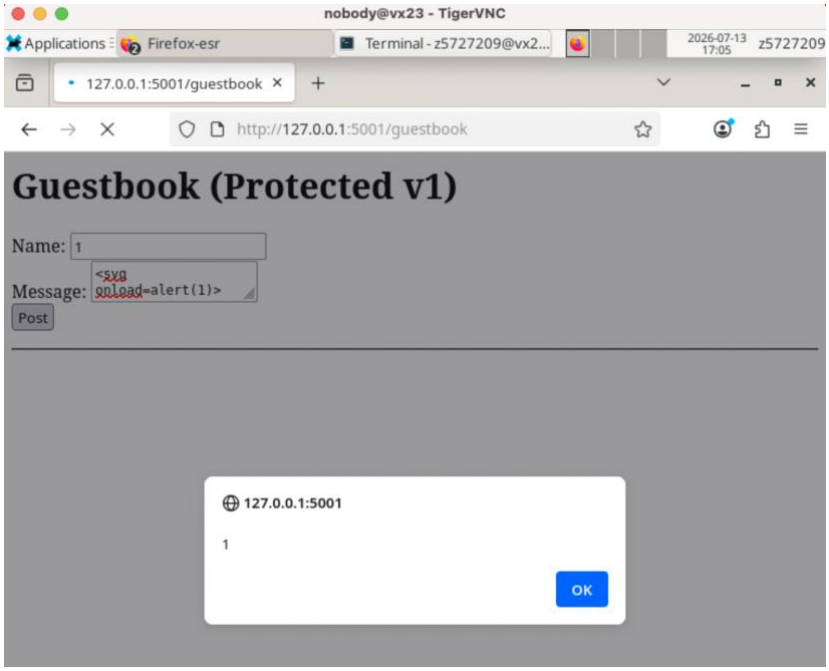

Figure 9: An <svg onload=...> payload never touches the tag-stripping filter and runs immediately.

3.2.3 Proper protection — output auto-escaping

The real fix takes | safe out entirely and lets Jinja2's default auto-escaping do its job:

```
{{ c[2] }}   <!-- previously {{ c[2] | safe }} -->
```

Comments are now stored exactly as typed, with no input filtering at all — the safety comes entirely from how they're displayed, not from anything done to the input beforehand. Posting the same <svg onload=alert(1)> payload that got past the weak filter just renders as plain, harmless text this time; the browser shows the angle brackets instead of interpreting them, because auto-escaping doesn't care what shape the attack takes.

Figure 10: The payload shows up as literal, harmless text instead of being executed.

3.2.4 A troubleshooting note

While going back and forth testing the two forms, I typed an XSS payload into the SQLi login form by mistake at one point. The app gave back a SQL syntax error instead of the 'blocked' message I was expecting, which threw me for a second, until I realised the payload hadn't actually been rejected by the blacklist at all — it didn't contain any of the blacklisted SQL keywords, so it was just meaningless SQL and the database itself threw the error. It was a genuinely useful reminder, even though it was an accident, that a filter only ever checks for exactly what it's told to check for, and an app can fail for completely different reasons even when the error on screen looks pretty similar.

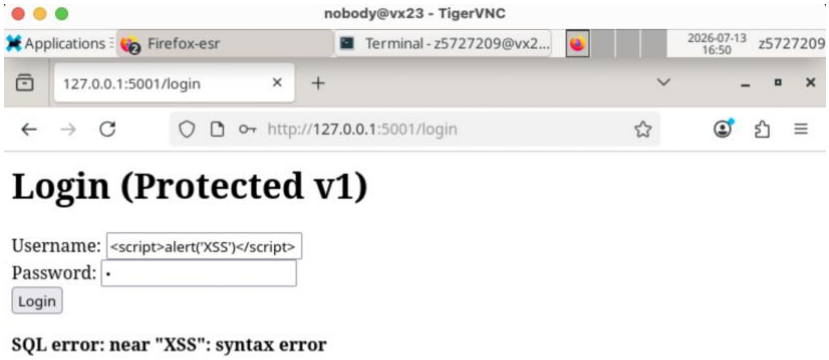

Figure 11: The SQL syntax error produced when an XSS payload got typed into the login field by mistake.

3.3 Binary Component — Buffer Overflow (ret2win)

3.3.1 Baseline — no protection

The vulnerable function reads user input into a fixed 64-byte stack buffer using read() with a size of 512 — way past what the buffer can actually hold — and never checks the length at all. There's a separate function, win(), that never gets called during normal program flow but does exist in the binary, and the goal was a classic ret2win attack: overflow the buffer far enough to overwrite the saved return address with win()'s address.

```
z5727209@vx15:~$ bash
z5727209@vx15:~$ cd ~/binary_bof
z5727209@vx15:~/binary_bof$ chmod +x build.sh
z5727209@vx15:~/binary_bof$ ./build.sh
Built ./vuln

Checking protections (if 'checksec' is installed):
(checksec not installed - that's fine, just proceed)
z5727209@vx15:~/binary_bof$ ./vuln
=== Baseline Buffer Overflow Challenge ===
What's your name? test
Hello, test
!
Goodbye!
z5727209@vx15:~/binary_bof$
```

Figure 12: The unmodified program compiles and behaves normally with ordinary input.

To find the exact offset to the saved return address, I sent a 200-byte cyclic (non-repeating) pattern, let the program crash, and used pwntools to parse the core dump and read the faulting address straight out of it.

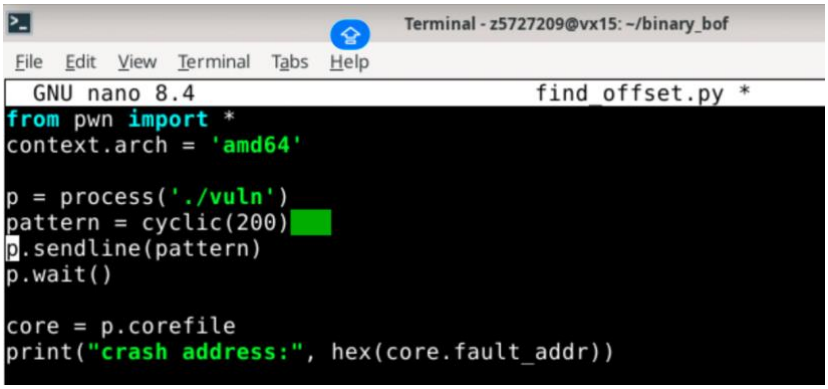

Figure 13: Script used to send a cyclic pattern and capture the crash.

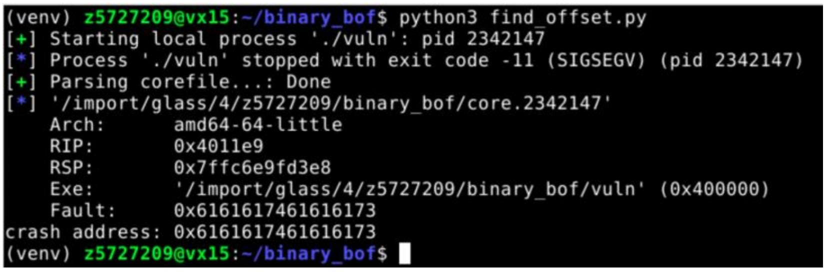

Figure 14: The program crashes trying to use a fragment of the pattern as an address — where that fragment sits in the original pattern gives the exact offset.

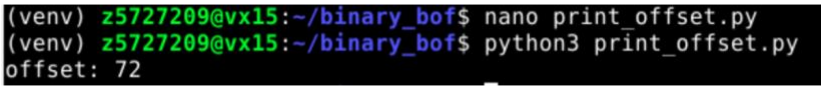

Figure 15: Offset confirmed as exactly 72 bytes.

Rather than hardcoding win()'s address by reading it off a disassembler, I pulled it straight from the ELF symbol table instead, so the exploit still works even if the binary gets recompiled and the address moves.

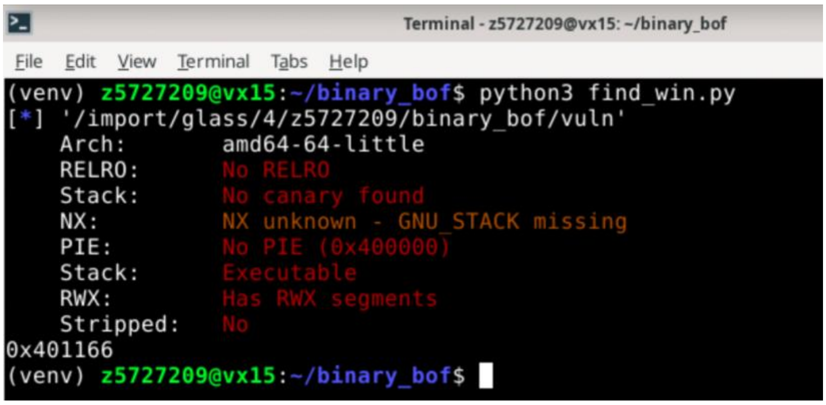

Figure 16: win()'s address (0x401166) read directly from the binary's symbol table, alongside checksec confirming no protections are active.

With both numbers known, the final payload is 72 bytes of padding followed by win()'s address written as an 8-byte little-endian pointer. Running this against the baseline binary redirects execution into win() and prints the flag.

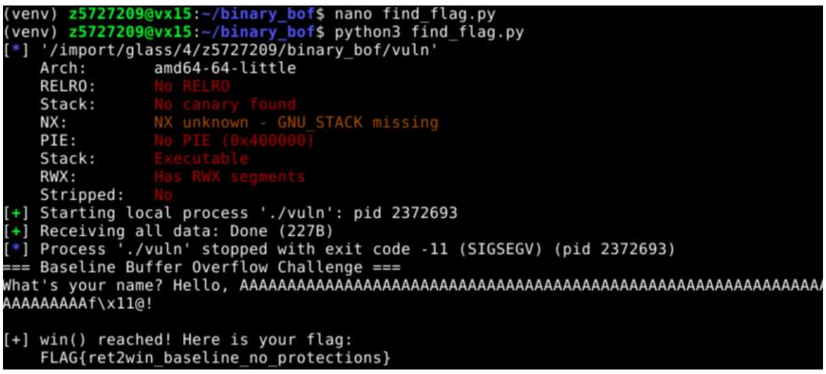

Figure 17: checksec confirms no protections are active; the crafted payload redirects execution into win(), printing the flag.

3.3.2 Weak protection — stack canary only

For the second version I only turned on the stack canary (compiled with -fstack-protector-all), leaving PIE, NX and RELRO off, so I could see exactly what the canary catches on its own. Running the

identical payload from Step 1 against this build, unmodified, now aborts with 'stack smashing detected' (SIGABRT) instead of reaching win().

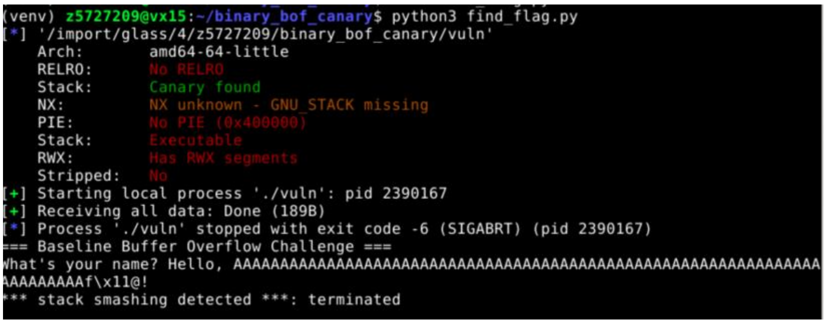

Figure 18: checksec confirms the canary is present; the same payload that worked in Step 1 is now caught before the function returns.

This shows the canary check happens structurally before the corrupted return address ever gets used — the exploit gets caught when the function exits, not at the moment the overflow actually happens.

3.3.3 Full protection — canary + PIE/ASLR + NX + RELRO

The third build just uses gcc's defaults with no special flags at all, which is basically what a modern Linux distribution compiles with normally. checksec confirms all four protections are on at the same time.

Figure 19: checksec confirms Full RELRO, a stack canary, NX, and PIE are all enabled together.

Because PIE randomises the load address every time the program runs, a fixed address like the one used in Step 1 can never reliably land on win(), even if the canary somehow got bypassed. I checked this directly with a script that starts the binary three times and reads its own base-address mapping out of /proc/pid/maps each time.

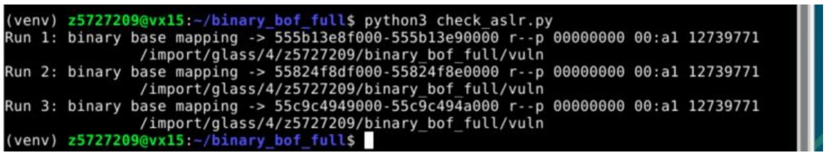

Figure 20: Three separate runs of the same binary load at three completely different base addresses, showing ASLR/PIE working.

In my testing, it was actually the canary that caught the reused Step 1 payload first, since it sits earlier in the stack layout than the return address itself — a pretty clean example of defence in depth. But even supposing an attacker got around the canary somehow, PIE would still block a naive hardcoded-address exploit, because there's just no way to know where win() sits in memory on any given run without a separate leak of some kind.

3.4 Binary Component — Format String Vulnerability

3.4.1 Baseline — no protection

The second program passes user input straight to printf() as the format string — printf(buf) instead of the safe printf("%s", buf). There's a global variable, secret_unlocked, that starts at 0, and the program only prints a flag if that value becomes non-zero. The challenge was to change it using

nothing but the format string bug — the read itself is properly bounded, so a buffer overflow isn't possible here at all.

I first checked the read side of the bug (the information leak) using repeated %x specifiers, which print raw stack and register values that were never supposed to be visible.

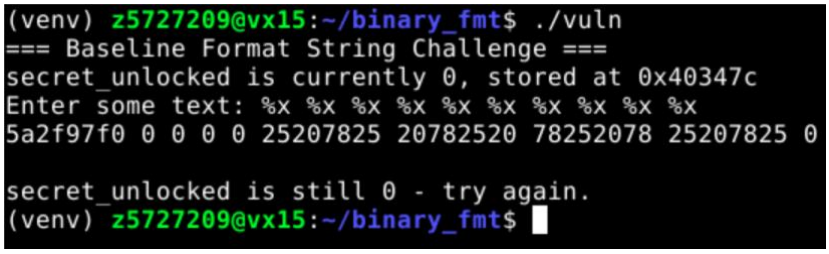

Figure 21: Repeated %x specifiers leak raw memory values off the stack, none of which I actually passed in as arguments.

Then I used positional parameter access (%N$x) to figure out which argument index lines up with my own input on the stack, testing each index by hand until a marker I'd typed got reflected back at me.

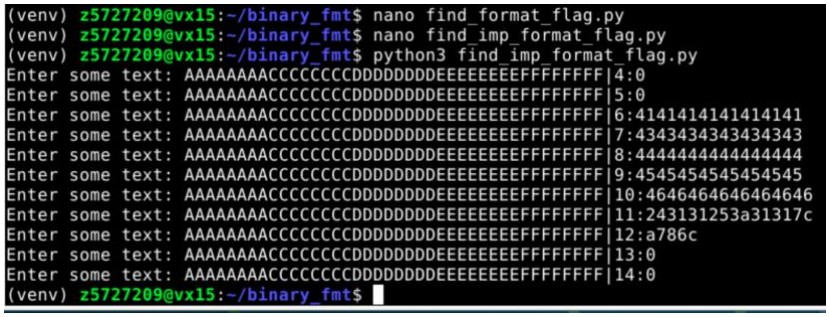

Figure 22: Scanning positional indices with a known marker ('MARK') shows index 6 reads back my own input, giving a usable position for the write primitive.

Using the write primitive (%n), which writes the number of characters printed so far into a pointer-sized argument, I built a payload that puts secret_unlocked's address at the exact stack offset that position 9 lines up with. A few literal characters before the %9$n directive make sure the value written isn't zero.

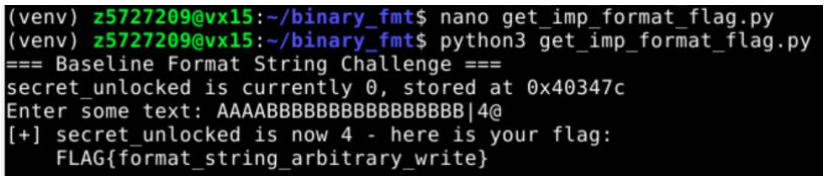

Figure 23: The crafted %n write changes secret_unlocked from 0 to 4, and the program prints the flag.

3.4.2 Weak protection — glibc _FORTIFY_SOURCE=2

The second build doesn't touch the code at all, it's just compiled with a different flag, -D_FORTIFY_SOURCE=2, which turns on glibc's runtime checks for certain risky library functions, printf included. Specifically, it notices when a format string sitting in writable memory (like our stack buffer) gets combined with %n, and kills the program instead of letting the write go ahead.

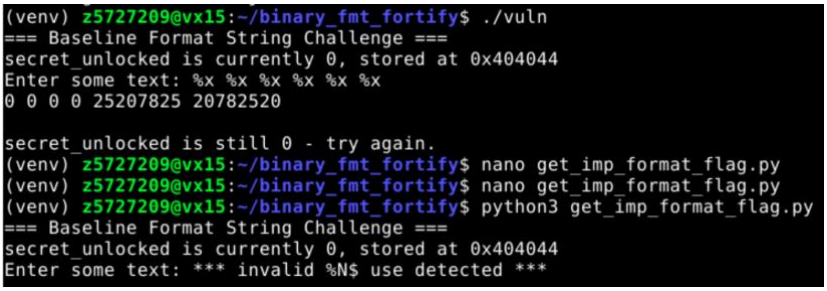

Figure 24: The %x read leak still works exactly as before (top), but the exact write payload now gets rejected with glibc's own 'invalid %N$ use detected' message (bottom).

This is a partial protection in more or less the same way the stack canary was for the buffer overflow — it's specifically aimed at the dangerous %n write, but does nothing about the %x/%p read primitives, which are still completely exploitable for leaking information.

3.4.3 Proper protection — source-level fix

The actual fix changes one line: printf(buf) becomes printf("%s", buf). The format string is now always the fixed literal "%s", and buf only ever gets treated as data, no matter what characters end up inside it.

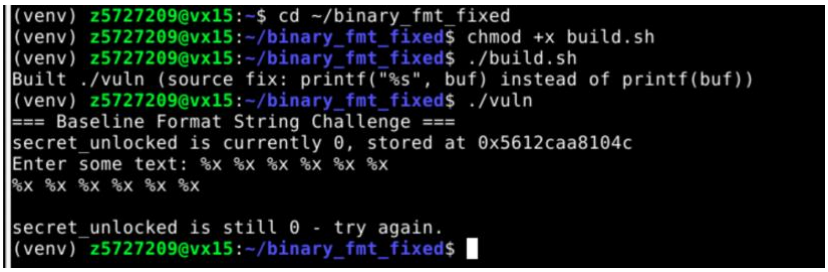

Figure 25: %x specifiers are no longer interpreted at all — they get printed back as the literal characters I typed.

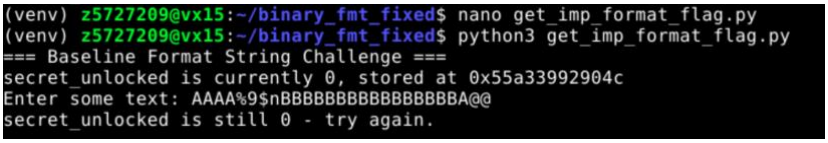

Figure 26: The full %n write payload also just gets printed back as harmless literal text; secret_unlocked stays 0.

Unlike the FORTIFY_SOURCE build, this fix gets rid of both the read and the write primitive at the same time, because it deals with the actual root cause — user input being parsed as instructions — instead of pattern-matching one dangerous specifier.

4. Results Summary

The table below pulls together every layer tested across both components. The pattern holds pretty consistently: denylist-style defences got bypassed, structural defences held.

Vulnerability	Baseline	Weak protection	Proper protection
SQL Injection	Bypassed — login skipped straight past authentication	Bypassed — worked around by changing case and comment style	Held — parameterised query
XSS (stored)	Bypassed — <script> runs immediately	Bypassed — <svg onload> never touched by the filter	Held — auto-escaping renders it as plain text

Buffer overflow (ret2win)	Bypassed — win() reached	Bypassed — canary only detects the crash, doesn't stop it happening	Held — PIE blocks the hardcoded address; canary also fires first
Format string	Bypassed — both the %x leak and the %n write work	Partly held — %n write blocked by FORTIFY_SOURCE, %x leak still works	Held — printf("%s", buf) removes both at once

5. Discussion and Reflection

5.1 Denylist vs. Structural Defence (Web)

Both weak-protection bypasses follow basically the same pattern even though SQLi and XSS are technically quite different bugs. The keyword blacklist and the tag-stripping filter are both denylists — they try to spot and reject inputs they already know are bad, so they're only as good as what their author happened to think of. My bypasses didn't need anything clever, just working out what the filter was literally comparing and finding an input with the same effect but different text. The proper fixes are structural instead: parameterised queries make it physically impossible for input to be interpreted as SQL syntax, and auto-escaping turns the characters a browser cares about into harmless text no matter what shape they arrive in. Neither needs to guess what an attacker might try next, which is exactly why both held up against the payloads that beat the blacklists.

5.2 Combining Vulnerability Classes (Binary)

There's a genuinely interesting connection between the two binary vulnerabilities beyond both just being 'binary exploitation'. A fully-protected buffer overflow target can't be exploited with a simple hardcoded ret2win payload, because there's no way to know where win() is loaded on a given run. A format string bug is exactly the kind of thing that could supply that missing piece — if the same binary had one elsewhere, an attacker could leak an address first, work out the runtime base address from it, and only then build a working ret2win payload. The canary would still need dealing with separately, but PIE stops being much of an obstacle once there's one reliable leak. I think this is a concrete example of why security engineering treats a system's vulnerabilities as a group rather than one at a time — a defence that looks complete against one isolated bug can be a lot weaker once a second, unrelated bug hands over the missing piece.

5.3 Cross-Cutting Comparison: Web vs. Binary

Put side by side, the web and binary halves reinforce the same lesson from two different technical areas. In both, the baseline vulnerability exists because user-supplied data gets treated as something more than plain data — a string parsed as SQL syntax, markup parsed as executable HTML, a format string parsed as printf instructions. In both, the weak protection targets the specific attack the developer had already seen rather than the capability underneath it, and the proper fix works by removing the confusion between code and data altogether. The binary side adds one wrinkle the web side doesn't have: because defences like PIE depend on the attacker being uncertain rather than removing a capability outright, one extra leak elsewhere in the same program can undo a protection that looked solid on its own.

5.4 Limits of the 'Proper' Fixes

Parameterised queries and auto-escaping are the right fixes for the bugs in this app, but neither one is a complete answer to injection in general, and I think that limitation matters just as much as the fix itself.

5.4.1 Limits of parameterised queries

Parameterisation only protects data values, not structural parts of a query like table names, column names, or ORDER BY direction, because SQL syntax doesn't allow those spots to be bound parameters at all. If an app let a user pick a sort column and that bit got built with string formatting 'just this once', the same class of bug would reappear in code that looks, at a glance, already fixed — sometimes called second-order or structural SQL injection.

5.4.2 Limits of auto-escaping

Jinja2's auto-escaping protects the specific context it was built for — plain HTML body text — but doesn't automatically protect every context, like an inline JavaScript block, a URL, or an unquoted attribute, which all care about different characters. It also only covers what the server renders, so it does nothing against DOM-based XSS, where client-side JavaScript shoves data straight into the page via element.innerHTML, sidestepping the template engine completely.

5.4.3 A defence-in-depth observation

While testing the fixed login form, Firefox's 'Save password' prompt showed the real admin credentials in plain text, because they're hardcoded in app.py rather than hashed in the database. Fixing the SQLi bug doesn't fix this — anyone with another way into the source would still get a working password instantly. A proper security review has to look at how data is protected at every stage, not just treat whichever bug got tested as the only one that matters.

5.5 Reflection: Challenges, Learning, and Professional Development

This project took roughly 30-40 hours altogether: setting up the CSE VLab environment, the exploitation work itself, debugging failed attempts, and writing up this report and the presentation.

The biggest technical struggle was the buffer overflow offset-finding process — getting a reliable, recompilable exploit (reading win()'s address off the ELF symbol table instead of hardcoding it) took longer than the initial crash did, but paid off when I needed the same logic to behave predictably across three compiler-flag setups. The format string write primitive was honestly the hardest part to wrap my head around — working out why %N$n needed a matching %N$x probe first, and why the count-so-far behaviour of %n meant padding mattered as much as the address itself, took a few failed attempts before it clicked.

Working on the web half brought a different challenge: resisting the urge to treat the weak-protection layer as 'basically secure' just because it blocked the first payload I tried. Asking what the filter was comparing, rather than what it was trying to achieve, was the real skill being practised, and it's the same skill behind both the SQLi and XSS bypasses despite them being unrelated bug classes. The late-night mistake of typing an XSS payload into the SQLi form (Section 3.2.4) was a small reminder to slow down and read error messages properly instead of assuming I already knew what they meant.

On the professional and ethical side, every vulnerable target here was written by me and only ever run on my own machine or the CSE VLab, never deployed anywhere publicly reachable — the goal was to understand these bugs safely, not to test systems I don't own. I also deliberately kept the weak-protection layer realistic (a blacklist and a stripped tag are genuine patterns in real, non-malicious code) rather than building something obviously flimsy, since the comparison only means anything if the weak version represents a real, plausible mistake.

Overall, this project changed how I think about 'is this actually fixed?'. Before it, a passing test against the one payload I knew about would have felt like proof. Now my instinct is to ask what class of input a fix actually removes, rather than what specific input it currently blocks, before calling anything resolved.

6. Conclusion

Across four vulnerability classes, spanning a web application and native binaries, the same lesson held every time: defences that recognise and reject specific known-bad inputs could be, and were, bypassed once I understood what comparison the filter was actually doing, while defences that remove the vulnerable operation entirely held up even against payloads built specifically to beat the weaker version. This project also showed that a 'proper' fix for one bug isn't the same as a secure system overall — parameterised queries and auto-escaping both have real, documented limits, and a fully-protected binary vulnerability can still be undermined by an unrelated leak elsewhere in the same program. Building, breaking, and then properly fixing my own vulnerable targets, rather than just reading about these bug classes, is what made the difference between 'looks fixed' and 'is fixed' feel concrete instead of theoretical.

7. References

OWASP Foundation (2021) OWASP Top 10:2021 - A03 Injection. Available at: https://owasp.org/Top10/2021/A03_2021-Injection/

OWASP Foundation (n.d.) SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation (n.d.) Cross Site Scripting Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

MITRE (2026) CWE-121: Stack-based Buffer Overflow. Available at: https://cwe.mitre.org/data/definitions/121.html

MITRE (2025) CWE-134: Use of Externally-Controlled Format String. Available at: https://cwe.mitre.org/data/definitions/134.html

Gallopeni, G. et al. pwntools documentation. Available at: https://docs.pwntools.com/

Pallets Projects (n.d.) Jinja2 template designer documentation - autoescaping. Available at: https://jinja.palletsprojects.com/en/stable/templates/#html-escaping

Chiu, E. (2026) comp6841-security-project [source code, screenshots, and evidence repository]. Available at: https://github.com/Amychiu1/comp6841-security-project

8. Appendix

8.1 Repository and Evidence

The full source code for every layer described in this report — all three web app.py versions, both vuln.c programs, the build scripts, and check_aslr.py — together with a full set of terminal and browser screenshots, is hosted at github.com/Amychiu1/comp6841-security-project. The repository README maps each file to the baseline/weak-protection/proper-protection layer it belongs to, for both the web and binary components, matching the section numbers used in this report. Key excerpts are reproduced below; the repository is the authoritative copy.

8.2 vuln.c — Buffer Overflow

Used, unmodified, across all three Buffer Overflow layers (Sections 3.3.1-3.3.3); only the compiler flags in build.sh changed between layers.

```
/*
 * Phase 1 - Binary Group - Step 1: Baseline Buffer Overflow
```

```
 * ============================================================
 * This program reads a name into a fixed-size stack buffer using an
 * unbounded read (gets-style). There is NO bounds checking at all.
 *
 * There is also a function, win(), that is never called by normal
 * program flow. Your goal is to overflow the buffer far enough to
 * overwrite the saved return address on the stack, so that when
 * greet() returns, execution jumps into win() instead of back to main().
 *
 * This is a classic "ret2win" challenge - a good first binary
 * exploitation exercise because you don't need to inject shellcode,
 * you just need to redirect control flow to code that already exists
 * in the binary.
 *
 * Compiled deliberately WITHOUT any modern protections - see build.sh
 */

#include <stdio.h>
#include <string.h>
#include <unistd.h>

void win() {
    printf("\n[+] win() reached! Here is your flag:\n");
    printf("    FLAG{ret2win_baseline_no_protections}\n\n");
}

void greet() {
    char name[64];
    printf("What's your name? ");
    fflush(stdout);

    // VULNERABLE: no bounds checking whatsoever. read() with a size far
    // larger than the buffer means we can write well past the end of
    // `name` and into the saved return address on the stack.
    read(0, name, 512);

    printf("Hello, %s!\n", name);
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("=== Baseline Buffer Overflow Challenge ===\n");
    greet();
    printf("Goodbye!\n");
    return 0;
}
```

8.3 vuln.c — Format String (baseline / FORTIFY_SOURCE layers)

Used, unmodified, in Sections 3.4.1 and 3.4.2; only the compiler flags changed between them. Section 3.4.3 changes a single line, printf(buf) to printf("%s", buf), as described above.

```
/*
 * Phase 1 - Binary Group - Format String - Step 1: Baseline
 * ============================================================
 * The program reads a line of input and passes it DIRECTLY to printf()
 * as the format string itself, instead of using it as data:
 *
 *     printf(buf);              // VULNERABLE
 *     instead of
 *     printf("%s", buf);        // safe - buf is just data here
 *
 * This means anything that looks like a format specifier (%x, %p, %n,
```

```
 * %s ...) inside YOUR input gets interpreted by printf as an instruction,
 * not printed as literal text.
 *
 * Two things become possible:
 *   1. READ: %x / %p let you read values off the stack that were never
 *      meant to be exposed (this is how you can leak addresses/canaries
 *      in a more advanced binary).
 *   2. WRITE: %n writes the number of characters printed SO FAR into an
 *      address you control - this is a full arbitrary-write primitive.
 *
 * Your goal: flip the global variable `secret_unlocked` from 0 to
 * non-zero using ONLY the format string bug (no buffer overflow is
 * needed or possible here - the buffer is read safely with a bounded
 * read()).
 */

#include <stdio.h>
#include <string.h>
#include <unistd.h>

int secret_unlocked = 0;   // global variable - lives at a fixed, known
                           // address because this binary is compiled
                           // without PIE (see build.sh)

void print_flag() {
    printf("[+] secret_unlocked is now %d - here is your flag:\n", secret_unlocked);
    printf("    FLAG{format_string_arbitrary_write}\n");
}

int main() {
    char buf[256];
    setvbuf(stdout, NULL, _IONBF, 0);

    printf("=== Baseline Format String Challenge ===\n");
    printf("secret_unlocked is currently %d, stored at %p\n", secret_unlocked,
(void*)&secret_unlocked);
    printf("Enter some text: ");

    memset(buf, 0, sizeof(buf));
    read(0, buf, sizeof(buf) - 1);

    printf(buf);   // <-- THE VULNERABILITY: user input used AS the format string

    printf("\n");
    if (secret_unlocked) {
        print_flag();
    } else {
        printf("secret_unlocked is still %d - try again.\n", secret_unlocked);
    }
    return 0;
}
```

8.4 app.py — Web Baseline

```
"""
Baseline Vulnerable Web App (Phase 1 - Step 1)
==============================================
Intentionally vulnerable Flask app for the COMP6841 security project.

Vulnerabilities included (NO protections yet - this is the baseline):
  1. SQL Injection in the /login route (raw string concatenation into SQL)
  2. Stored XSS in the /guestbook route (user input rendered without escaping)
```

```
Run:
    pip install flask
    python3 app.py

Then visit http://127.0.0.1:5000/
"""

from flask import Flask, request, render_template_string, g
import sqlite3
import os

app = Flask(__name__)
DB_PATH = os.path.join(os.path.dirname(__file__), "vuln.db")



def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
    return g.db



@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()



def init_db():
    # Fresh DB every time the app starts, so your testing is repeatable
    if os.path.exists(DB_PATH):
        os.remove(DB_PATH)
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("""CREATE TABLE users (
                        id INTEGER PRIMARY KEY,
                        username TEXT,
                        password TEXT
                    )""")
    cur.execute("""CREATE TABLE comments (
                        id INTEGER PRIMARY KEY,
                        author TEXT,
                        body TEXT
                    )""")
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("admin", "S3cretAdminPassw0rd!"))
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("alice", "alice123"))
    conn.commit()
    conn.close()



# ---------- HOME ----------
HOME_HTML = """
<h1>Vulnerable Demo App - Phase 1 Baseline</h1>
<ul>
  <li><a href="/login">Login page (SQLi target)</a></li>
  <li><a href="/guestbook">Guestbook (XSS target)</a></li>
</ul>
"""
```

```
@app.route("/")
def home():
    return HOME_HTML



# ---------- LOGIN (SQL INJECTION) ----------
LOGIN_HTML = """
<h1>Login</h1>
<form method="POST">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
{% if message %}<p><strong>{{ message }}</strong></p>{% endif %}
"""



@app.route("/login", methods=["GET", "POST"])
def login():
    message = None
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")

        # VULNERABLE: raw string concatenation, no parameterisation.
        # This is the exact line you will screenshot and analyse.
        query = "SELECT * FROM users WHERE username = '{}' AND password = '{}'".format(
            username, password
        )

        db = get_db()
        cur = db.cursor()
        try:
            cur.execute(query)
            row = cur.fetchone()
        except sqlite3.OperationalError as e:
            row = None
            message = "SQL error (useful for error-based injection): {}".format(e)

        if row:
            message = "Login successful! Welcome, {}".format(row[1])
        elif not message:
            message = "Login failed."

    return render_template_string(LOGIN_HTML, message=message)



# ---------- GUESTBOOK (STORED XSS) ----------
GUESTBOOK_HTML = """
<h1>Guestbook</h1>
<form method="POST">
  Name: <input type="text" name="author"><br>
  Message: <textarea name="body"></textarea><br>
  <input type="submit" value="Post">
</form>
<hr>
{% for c in comments %}
  <p><strong>{{ c[1] | safe }}</strong> says: {{ c[2] | safe }}</p>
{% endfor %}
"""
```

```
@app.route("/guestbook", methods=["GET", "POST"])
def guestbook():
    db = get_db()
    cur = db.cursor()

    if request.method == "POST":
        author = request.form.get("author", "")
        body = request.form.get("body", "")
        # VULNERABLE: stored directly, rendered later with | safe (no escaping)
        cur.execute("INSERT INTO comments (author, body) VALUES (?, ?)", (author, body))
        db.commit()

    cur.execute("SELECT * FROM comments ORDER BY id DESC")
    comments = cur.fetchall()
    return render_template_string(GUESTBOOK_HTML, comments=comments)


if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=True)
```

8.5 app_weak_protection.py — Weak Protection

```
"""
Phase 1 - Step 2: "Weak Protection" Version
============================================
This version adds NAIVE protections - the kind a junior developer might add
without fully understanding the vulnerability class. Your job in this step
is to try to BYPASS these protections and document what works / what doesn't.

Protections added:
  1. SQLi: blacklist filter that rejects input containing certain keywords
  2. XSS: blacklist filter that strips <script> tags (naively)

These are DELIBERATELY incomplete - that's the point of this exercise.
Run:
    python3 app_weak_protection.py
"""

from flask import Flask, request, render_template_string, g
import sqlite3
import os

app = Flask(__name__)
DB_PATH = os.path.join(os.path.dirname(__file__), "vuln.db")


def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
    return g.db


@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()
```

```
def init_db():
    if os.path.exists(DB_PATH):
        os.remove(DB_PATH)
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("""CREATE TABLE users (
                        id INTEGER PRIMARY KEY,
                        username TEXT,
                        password TEXT
                    )""")
    cur.execute("""CREATE TABLE comments (
                        id INTEGER PRIMARY KEY,
                        author TEXT,
                        body TEXT
                    )""")
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("admin", "S3cretAdminPassw0rd!"))
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("alice", "alice123"))
    conn.commit()
    conn.close()


HOME_HTML = """
<h1>Vulnerable Demo App - "Weak Protection" Layer</h1>
<ul>
  <li><a href="/login">Login page (SQLi - naive keyword blacklist)</a></li>
  <li><a href="/guestbook">Guestbook (XSS - naive tag stripping)</a></li>
</ul>
"""



@app.route("/")
def home():
    return HOME_HTML



# ---------- LOGIN with NAIVE SQLi "protection" ----------
LOGIN_HTML = """
<h1>Login (Protected v1)</h1>
<form method="POST">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
{% if message %}<p><strong>{{ message }}</strong></p>{% endif %}
"""

# Naive blacklist: case-sensitive, only checks a few exact keywords.
# Think about how you would defeat this before reading further.
SQL_BLACKLIST = ["OR", "UNION", "--", "DROP", "SELECT"]



def naive_sql_filter(value):
    for bad_word in SQL_BLACKLIST:
        if bad_word in value:
            return False
    return True



@app.route("/login", methods=["GET", "POST"])
```

```
def login():
    message = None
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")

        if not naive_sql_filter(username) or not naive_sql_filter(password):
            message = "Blocked: suspicious input detected."
        else:
            # Still vulnerable underneath - the ONLY defence is the blacklist above.
            query = "SELECT * FROM users WHERE username = '{}' AND password = '{}'".format(
                username, password
            )
            db = get_db()
            cur = db.cursor()
            try:
                cur.execute(query)
                row = cur.fetchone()
            except sqlite3.OperationalError as e:
                row = None
                message = "SQL error: {}".format(e)

            if row:
                message = "Login successful! Welcome, {}".format(row[1])
            elif not message:
                message = "Login failed."

    return render_template_string(LOGIN_HTML, message=message)



# ---------- GUESTBOOK with NAIVE XSS "protection" ----------
GUESTBOOK_HTML = """
<h1>Guestbook (Protected v1)</h1>
<form method="POST">
  Name: <input type="text" name="author"><br>
  Message: <textarea name="body"></textarea><br>
  <input type="submit" value="Post">
</form>
<hr>
{% for c in comments %}
  <p><strong>{{ c[1] | safe }}</strong> says: {{ c[2] | safe }}</p>
{% endfor %}
"""



def naive_xss_filter(value):
    # Non-recursive, case-sensitive removal of exact "<script>" / "</script>".
    # Think about what payloads this does NOT catch.
    value = value.replace("<script>", "").replace("</script>", "")
    return value



@app.route("/guestbook", methods=["GET", "POST"])
def guestbook():
    db = get_db()
    cur = db.cursor()

    if request.method == "POST":
        author = request.form.get("author", "")
        body = request.form.get("body", "")
        body = naive_xss_filter(body)
        cur.execute("INSERT INTO comments (author, body) VALUES (?, ?)", (author, body))
        db.commit()
```

```
    cur.execute("SELECT * FROM comments ORDER BY id DESC")
    comments = cur.fetchall()
    return render_template_string(GUESTBOOK_HTML, comments=comments)


if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5001, debug=True)
```

8.6 app_fixed.py — Proper Protection

```
"""
Phase 1 - Step 3: "Proper Protection" Version
================================================
This version replaces the naive blacklist approach with industry-standard
defences:

  1. SQLi -> Parameterised queries (data is never concatenated into SQL text;
     the DB driver keeps code and data completely separate)
  2. XSS  -> Output auto-escaping (Jinja2's default behaviour - we simply
     REMOVED the `| safe` filter that was disabling it. No input filtering
     needed at all.)

Try the exact same bypass payloads that worked in Step 2 and see what
happens now.

Run:
    python3 app_fixed.py
"""

from flask import Flask, request, render_template_string, g
import sqlite3
import os

app = Flask(__name__)
DB_PATH = os.path.join(os.path.dirname(__file__), "vuln.db")


def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
    return g.db


@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()


def init_db():
    if os.path.exists(DB_PATH):
        os.remove(DB_PATH)
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("""CREATE TABLE users (
                        id INTEGER PRIMARY KEY,
                        username TEXT,
                        password TEXT
```

```
                    )""")
    cur.execute("""CREATE TABLE comments (
                        id INTEGER PRIMARY KEY,
                        author TEXT,
                        body TEXT
                    )""")
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("admin", "S3cretAdminPassw0rd!"))
    cur.execute("INSERT INTO users (username, password) VALUES (?, ?)",
                ("alice", "alice123"))
    conn.commit()
    conn.close()


HOME_HTML = """
<h1>Vulnerable Demo App - "Proper Protection" Layer</h1>
<ul>
  <li><a href="/login">Login page (SQLi - parameterised query)</a></li>
  <li><a href="/guestbook">Guestbook (XSS - auto-escaped output)</a></li>
</ul>
"""



@app.route("/")
def home():
    return HOME_HTML



# ---------- LOGIN with PARAMETERISED QUERY ----------
LOGIN_HTML = """
<h1>Login (Fixed)</h1>
<form method="POST">
  Username: <input type="text" name="username"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
{% if message %}<p><strong>{{ message }}</strong></p>{% endif %}
"""



@app.route("/login", methods=["GET", "POST"])
def login():
    message = None
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")

        # FIX: username/password are passed as bound parameters (the "?"
        # placeholders). SQLite treats them purely as DATA - they can never
        # be interpreted as part of the SQL command, no matter what
        # characters they contain.
        query = "SELECT * FROM users WHERE username = ? AND password = ?"

        db = get_db()
        cur = db.cursor()
        try:
            cur.execute(query, (username, password))
            row = cur.fetchone()
        except sqlite3.OperationalError as e:
            row = None
            message = "SQL error: {}".format(e)

        if row:
```

```
            message = "Login successful! Welcome, {}".format(row[1])
        elif not message:
            message = "Login failed."

    return render_template_string(LOGIN_HTML, message=message)



# ---------- GUESTBOOK with AUTO-ESCAPED OUTPUT ----------
# NOTE the `| safe` filters from the previous versions are GONE here.
# Jinja2 escapes HTML-special characters by default on every {{ }} - this
# is the fix. No input filtering / sanitisation needed.
GUESTBOOK_HTML = """
<h1>Guestbook (Fixed)</h1>
<form method="POST">
  Name: <input type="text" name="author"><br>
  Message: <textarea name="body"></textarea><br>
  <input type="submit" value="Post">
</form>
<hr>
{% for c in comments %}
  <p><strong>{{ c[1] }}</strong> says: {{ c[2] }}</p>
{% endfor %}
"""



@app.route("/guestbook", methods=["GET", "POST"])
def guestbook():
    db = get_db()
    cur = db.cursor()

    if request.method == "POST":
        author = request.form.get("author", "")
        body = request.form.get("body", "")
        # Stored EXACTLY as typed - no input filtering at all.
        # Safety comes entirely from output-side escaping above.
        cur.execute("INSERT INTO comments (author, body) VALUES (?, ?)", (author, body))
        db.commit()

    cur.execute("SELECT * FROM comments ORDER BY id DESC")
    comments = cur.fetchall()
    return render_template_string(GUESTBOOK_HTML, comments=comments)



if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5002, debug=True)
```